COMP 018: Database Administration - Activity 5 Progress Report
Database Maintenance, Performance Monitoring, and Query Analysis

Project Identification & Metadata

Course	: COMP 018 - Database Administration
Evaluating Group	: Group 4: Olivarez, Abangan, Pocsidio, Infantado, and Bartolata
Evaluating Group Repository	: https://github.com/Nycri/brewpos
Target Project Monitored	: Park-It: Parking Management System (Park-It Database Management System)
Target Project Group	: Group 2: Miller Devera, Jhon Francis, et al.
Target Project Repository	: https://github.com/millerdevera6-glitch/park-it-database
Primary Database	: grp_parkitdb (PostgreSQL 18.6 on x86_64 Windows, compiled by MSVC)
Restored Test Database	: grp_parkitdb_restore_test

ACTIVITY 5: Database Maintenance and Monitoring

2.1 Database Health & Overview Inspection
Database health inspection was conducted on PostgreSQL 18.6 running on a 64-bit Windows environment. Analysis confirmed that the database occupies a total disk footprint of approximately 8.4 megabytes across five base tables and five sequences within the public schema. Table storage analysis showed that parking zone, staff user, vehicle, and payment tables occupy 40 kilobytes each, while transaction records occupy 24 kilobytes.

2.2 Connection, Session & Activity Monitoring
Database statistics recorded four active connections with 1,205 committed transactions and 13 rolled back transactions. Zero lock conflicts and zero deadlocks were recorded across sessions, confirming high transaction integrity with a rollback ratio below 1.07 percent. System catalog queries verified that active roles and memberships accurately reflect access control rules.

2.3 Table Statistics & Bloat Inspection
Pre-maintenance table statistics revealed dead tuple accumulation within the parking zone table resulting from insertion, update, and deletion operations executed during Activity 4 verification testing. Remaining operational tables exhibited zero dead tuples. The presence of obsolete row versions confirmed the necessity of performing routine tuple vacuuming and statistics updating operations.

2.4 Log Review & Issue Identification
Review of database logs verified that unauthorized access attempts during Activity 4 security testing were recorded by the engine. Logged events included permission denials on deletion commands attempted on transaction and staff tables, denied query attempts on credential tables, blocked zone insertions by reporting accounts, rejected database creation requests, and

denied cross-database connection attempts. Background engine processes logged clean checkpoint completions without crashes or storage warnings.

2.5 Routine Maintenance Operations
Routine maintenance was conducted by executing system statistics updating and tuple vacuuming operations across target tables. Initial permission testing confirmed that non-owner application runtime accounts are denied execution of maintenance commands, proving that maintenance tasks require administrative credentials.

Full database statistics updates executed cleanly under administrative credentials. Standard vacuuming on the parking zone table scanned the relation, removing obsolete row versions and updating frozen transaction counts. Vacuum processing on transaction and payment tables verified table health and refreshed statistics without locking issues.

Full table rewriting vacuum operations were omitted because such commands acquire exclusive locks that block concurrent application read and write operations. Because relations reside within single disk pages, standard non-blocking vacuuming reclaimed dead tuples effectively without disrupting application availability.

2.6 Query Performance Analysis
Performance analysis was conducted on a parking zone capacity lookup query. The query executed in 0.050 milliseconds with complete buffer cache hit efficiency. The query planner selected a sequential scan over an index scan because the relation fits within a single memory buffer block, avoiding index page processing overhead.

Analysis of a vehicle stay and payment history query joining transaction, vehicle, parking zone, and payment tables yielded an execution time of 0.378 milliseconds with zero physical disk reads. The query engine built in-memory hash tables across join relations, executing chained hash joins within dedicated memory structures without disk spills.

2.7 Prioritized Recommendations & Technical Justification
The current index configuration is appropriate for operational requirements, as active queries execute under 0.38 milliseconds with complete buffer cache hits. Immediate index additions are unnecessary and would introduce write overhead during record updates. If transaction volume increases significantly in production, adding a composite index covering zone identifiers and entry timestamps will optimize vehicle tracking queries. Deploying a scheduled background maintenance task will maintain page efficiency during off-peak hours.

CLASS GOOGLE SHEETS TRACKER ENTRY (Activity 5)

Column Header	Value
Project	Group 2 - Park-It: Parking Management System
Database inspected	grp_parkitdb (PostgreSQL 18.6)
Account and role used	postgres (DBA Maintenance), parkit_app (Runtime), parkit_report_user (Audit)
Health check result	Healthy (0 conflicts, 0 deadlocks, 8.4 MB footprint, 100% buffer cache hits)
Maintenance performed	ANALYZE; VACUUM (VERBOSE, ANALYZE) on parking_zone, transaction, payment via postgres
Query analyzed	Parking zone occupancy lookup (Seq scan, 0.050 ms, 4 buffer hits) & Multi-table vehicle stay join (Hash joins, 0.378 ms)
Recommendation priority	No action required (Current queries execute in < 0.38 ms with 100% buffer cache hits)
Evidence folder link	https://github.com/Nycri/brewpos/tree/main/activity-5/Evidence-act5
Feedback sent to owners	(05/11/2026)
PDF progress document	Activity5_Progress_Report_Group2.pdf
Last Updated	2026-11-05

AI USAGE DECLARATION

Google Gemini 3.8 Flash was utilized during project evaluation to inspect database schema definitions, evaluate relational hierarchy structures, formulate role verification test scenarios, analyze database maintenance operations and query performance execution plans, verify backup creation and restoration procedures, and structure technical report documentation. All database statements, configuration checks, and verification procedures were directly executed and validated against PostgreSQL 18.6 within the local database environment.